Source Code Screenshot:

daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)

File Edit Format Run Options Window Help

```
import time
import random
import statistics
import csv
import matplotlib.pyplot as plt
SIZES = [1000, 10000, 100000, 1000000]
REQUESTS = 30
# 1. LINEAR SEARCH
def linear_search(data, target):
    comparisons = 0
    for i in range(len(data)):
        comparisons += 1
        if data[i] == target:
            return i, comparisons
    return -1, comparisons
# 2. ITERATIVE BINARY SEARCH
def binary_search(data, target):
    low = 0
    high = len(data) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if data[mid] == target:
            return mid, comparisons
        elif data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons
# 3. RECURSIVE BINARY SEARCH
def recursive_binary_search(data, target, low=0, high=None,
                            comparisons=None):
    if high is None:
        high = len(data) - 1
    if comparisons is None:
        comparisons = [0]
    if low > high:
        return -1, comparisons[0]
    mid = (low + high) // 2
    comparisons[0] += 1
    if data[mid] == target:
        return mid, comparisons[0]
    elif data[mid] < target:
        return recursive_binary_search(
            data, target, mid + 1, high, comparisons
        )
    else:
        return recursive_binary_search(
```

daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)

File Edit Format Run Options Window Help

```
            data, target, low, mid - 1, comparisons
        )
# MEASURE EXECUTION TIME
def measure(algorithm, data, target):
    start = time.perf_counter()
    result, comparisons = algorithm(data, target)
    end = time.perf_counter()
    execution_time = (end - start) * 1000
    return result, comparisons, execution_time
# MAIN PROGRAM
def main():
    random.seed(42)
    results = []
    algorithms = [
        ("Linear Search", linear_search),
        ("Iterative Binary Search", binary_search),
        ("Recursive Binary Search", recursive_binary_search)
    ]
    for n in SIZES:
        print("\nDataset size:", n)
        # Generate unique student registration numbers
        sorted_data = list(range(10000000, 10000000 + n))
        # Shuffle data for linear search
        unsorted_data = sorted_data.copy()
        random.shuffle(unsorted_data)
        targets = [
            random.choice(sorted_data)
            for _ in range(REQUESTS)
        ]
        for name, algorithm in algorithms:
            # Binary search needs sorted data
            if "Binary" in name:
                data = sorted_data
            else:
                data = unsorted_data
            times = []
            comparisons_list = []
            for target in targets:
                index, comparisons, elapsed = measure(
                    algorithm, data, target
                )
                if index == -1 or data[index] != target:
                    raise ValueError("Search result is incorrect")
                times.append(elapsed)
                comparisons_list.append(comparisons)
            avg_time = statistics.mean(times)
            avg_comparisons = statistics.mean(comparisons_list)
            results.append({
                "Dataset Size": n,
                "Algorithm": name,
```

Ln: 167 Col: 71

daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)

File Edit Format Run Options Window Help

```
            "Maximum Comparisons"
        ]
        writer = csv.DictWriter(file, fieldnames=fields)
        writer.writeheader()
        writer.writerows(results)
    # GRAPH 1: EXECUTION TIME
    plt.figure()
    for name, _ in algorithms:
        rows = [r for r in results if r["Algorithm"] == name]
        plt.plot(
            [r["Dataset Size"] for r in rows],
            [r["Average Time (ms)"] for r in rows],
            marker="o",
            label=name
        )
    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Number of Student Records")
    plt.ylabel("Average Execution Time (ms)")
    plt.title("Search Algorithm Execution Time")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("execution_time.png")
    plt.show()
    # GRAPH 2: NUMBER OF COMPARISONS
    plt.figure()
    for name, _ in algorithms:
        rows = [r for r in results if r["Algorithm"] == name]
        plt.plot(
            [r["Dataset Size"] for r in rows],
            [r["Average Comparisons"] for r in rows],
            marker="o",
            label=name
        )
    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Number of Student Records")
    plt.ylabel("Average Number of Comparisons")
    plt.title("Search Algorithm Comparison Count")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("key_comparisons.png")
    plt.show()
    print("\nAll search tests passed.")
    print("Results saved to benchmark_results.csv")
    print("Graphs saved to execution_time.png and key_comparisons.png")
if __name__ == "__main__":
    main()
```

```
# Comparison count graph
for name in algorithms:
    plt.plot(sizes, comparisons[name], marker="o", label=name)

plt.xscale("log")
plt.yscale("log")
plt.xlabel("Number of Student Records")
plt.ylabel("Number of Comparisons")
plt.title("Comparison Count Analysis")
plt.legend()
plt.grid(True)
plt.savefig("comparisons.png")
plt.show()

print("\nAll experiments completed.")
print("Graphs saved successfully.")
# GRAPH 2: NUMBER OF COMPARISONS

plt.figure()

for name in algorithms:
    plt.plot(
        sizes,
        comparisons[name],
        marker="o",
        label=name
    )
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Number of Student Records")
plt.ylabel("Number of Comparisons")
plt.title("Comparison Count Analysis of Searching Algorithms")
plt.legend()
plt.grid(True)
plt.savefig("comparisons.png")
plt.show()
print("Comparison graph generated successfully!")
```

```
# TASK 9: SAVE EXPERIMENTAL RESULTS IN CSV

import csv

with open("performance_results.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow([
        "Dataset Size",
        "Algorithm",
        "Execution Time (ms)",
        "Number of Comparisons"
    ])

    for name in algorithms:
        for i in range(len(sizes)):
            writer.writerow([
                sizes[i],
                name,
                round(times[name][i], 6),
                comparisons[name][i]
            ])

print("Performance results saved successfully!")
print("File created: performance_results.csv")
```